ME 3204 Mechatronics

Mini Rocket Fin Control System

Design Project Report



1 System Description

My Mini Rocket Fin Control System is a scaled down model that uses the changing of the orientation of the airframe to move four rocket fins. An MPU-6050 IMU measures acceleration and angular velocity, and the Arduino uses that data to estimate pitch and roll. The Arduino then sends commands to a PCA9685 servo driver, which moves four servos attached to fins. The LCD displays live values during testing.

MPU-6050 IMU → Arduino → PCA9685 servo driver → servos → fins

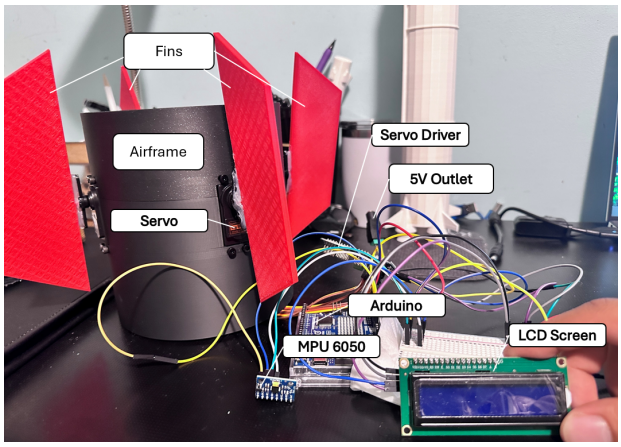


Figure 1: Final Mini Rocket Fin Control System showing the IMU input sensor, Arduino controller, LCD, PCA9685 servo driver, servo actuators, fins, and external servo power supply input which is only connected to the servo driver.

The input sensor is a MPU-6050 IMU. The output actuators are the four servos, and the output mechanism is the set of four fins attached to servo horns built for these CN0023 servos. The project combines sensing, Arduino processing, PWM servo control, LCD feedback, and 3D printed construction into one working system.

2 System Components, Integration, and Construction

The mechanical system was built around a 3D printed airframe section with 4 3D printed fins arranged around it. Each fin is attached to a servo horn, and each horn is connected to a servo shaft. The fins and horns were glued together so the fins would not slip during motion. This matters because the code assumes that the commanded servo angle matches the actual fin angle. In other words, the fins and servo horns had to be glued together otherwise trying to get a physical idea of where the fins should be would be impossible. The measured fin face area was $6619.915\,\text{mm}^2$, and the distance from the airframe center of mass to the center of mass of the fin was $106.961\,\text{mm}$.

The electronic system is controlled by an Arduino UNO R4. The MPU-6050 sends acceleration and angular velocity data to the Arduino communicating through I2C. The LCD and PCA9685 servo driver also use I2C. On the Arduino, A4 is SDA and A5 is SCL. The MPU-6050 is powered from 3.3 V, while the LCD and PCA9685 logic side are powered from 5 V. The servos are powered by an external 5 V, 10 A supply because the Arduino cannot safely power multiple servos at once. The Arduino and external power supply share a common ground. The PCA9685 servo driver creates the PWM signals for the four servos. Channel 0 controls the top fin, channel 1 controls the right fin, channel 2 controls the bottom fin,

and channel 3 controls the left fin. The LCD displays live pitch, roll, servo angles, gain, deadband, range, and trim values.

The MPU-6050 contains an accelerometer and a gyroscope. The accelerometer measures acceleration along three axes, which lets the Arduino estimate tilt using gravity. A gyroscope measures angular velocity, or how fast the sensor is rotating. The gyroscope gives smooth short-term motion, but it can slowly drift over time. During startup, the IMU is held still so the code can estimate gyro bias. This bias is subtracted from future gyro readings to reduce drift in the pitch and roll estimate.

The Arduino estimates pitch and roll using a complementary filter. A complementary filter is a simple way to combine two imperfect sensor measurements. In this project, the accelerometer gives a long-term tilt reference, and the gyroscope gives smoother short term motion. The filter combines both so the angle estimate is smoother than using the accelerometer alone and less likely to drift than using the gyroscope alone.

$$\text{roll}_{accel} = \tan^{-1}\left(\frac{a_y}{a_z}\right) \qquad \text{pitch}_{accel} = \tan^{-1}\left(\frac{-a_x}{\sqrt{a_y^2 + a_z^2}}\right) \tag{1}$$

$$\text{roll} = \alpha(\text{roll} + g_x \Delta t) + (1-\alpha)\text{roll}_{accel} \tag{2}$$

$$\text{pitch} = \alpha(\text{pitch} + g_y \Delta t) + (1-\alpha)\text{pitch}_{accel} \tag{3}$$

In the code, $\alpha = 0.97$. This value was selected experimentally to balance smooth gyroscope-based response with accelerometer correction. The gyroscope helps the fins respond smoothly when the model is tilted, but small gyro errors can build up over time and cause drift. The accelerometer helps correct this because it senses the direction of gravity. Using $\alpha = 0.97$ means the code mostly follows the gyroscope for smooth motion, while still using a small amount of accelerometer correction to keep the pitch and roll estimate accurate over time. The code also uses a small deadband and smoothing so tiny IMU noise does not make the fins twitch near level.

The fin control is proportional, meaning a larger tilt causes a larger fin movement. The gain $K = 1.8$ was selected experimentally so the fins moved noticeably in response to tilt without immediately hitting their motion limits. For example, if the model tilts by 10°, the code commands about 18° of fin correction because $1.8 \times 10 = 18$. The servos are centered at 90°, so the code adds or subtracts the correction from 90°. Opposite fins move in opposite directions so they act as a correcting pair.

$$\text{top fin} = 90^\circ + K\theta \qquad \text{bottom fin} = 90^\circ - K\theta \tag{4}$$

$$\text{right fin} = 90^\circ + K\phi \qquad \text{left fin} = 90^\circ - K\phi \tag{5}$$

Here, θ is pitch and ϕ is roll. The top and bottom fins respond to pitch, while the right and left fins respond to roll. The servo range is limited from 50° to 130°, and trim values are used to straighten the fins at rest.

In this project, it connects to the topics we talked about in class such as sensors, interface electronics, actuators, and PWM control. The MPU-6050 is the sensor input, the Arduino processes the signal, the PCA9685 creates PWM commands, and the servos convert those commands into fin motion. The final system combines sensing, processing, power distribution, display feedback, and actuator motion.

3 New Elements in the System

This project should have a high complexity rating because it combines multiple sensors, interface devices, actuators, power connections, and software into a system. The system had to read orientation data, process it, display live values, and safely move four fins at the same time.

- **MPU-6050 IMU integration:** The system uses acceleration and angular velocity data to estimate pitch and roll instead of using a simple position sensor. This required integrating a new sensor and using its data as the main input for the system.
- **Complementary filter:** The code combines accelerometer and gyroscope data to create a cleaner angle estimate. This added more complex code because the Arduino had to process sensor data before commanding the fins.
- **PCA9685 servo driver:** A dedicated PWM driver controls the servos instead of relying only on Arduino PWM pins. This made it possible to control multiple servos through one interface while keeping the wiring and control logic more organized.
- **LCD and Serial Monitor interface:** The LCD displays live system values, and the Serial Monitor allows testing and calibration during operation. This made the system easier to debug and helped verify that the sensor readings and fin commands were working correctly.

A Parts List and Reference Links

Table 1: Main parts used in the Mini Rocket Fin Control System.

Part	Purpose	Reference Link
Arduino Uno R4 WiFi	Main controller	Arduino Uno R4 WiFi
MPU-6050 GY-521 IMU	Acceleration and angular velocity sensing	MPU-6050 GY-521 IMU
MPU-6050 product specification	Main IMU datasheet	MPU-6050 product specification
MPU-6050 register map	Supporting register reference	MPU-6050 register map
PCA9685 servo driver	Servo PWM control	PCA9685 servo driver
SunFounder CN0023 servo	Fin actuation	SunFounder CN0023 servo
I2C 16 by 2 LCD screen	System feedback display	GeeekPi I2C LCD
External 5 V, 10 A supply	Servo power	5 V 10 A power supply
3D printed airframe and fins	Main structure and control surfaces	Fusion 360 airframe and fin model
GitHub repository	Final code and project files	RocketFinControlSystem GitHub rep

B Detailed Wiring Reference

Table 2: Main I2C wiring.

Connection	Purpose
Arduino A4 to LCD SDA, PCA9685 SDA, and MPU-6050 SDA	I2C data line
Arduino A5 to LCD SCL, PCA9685 SCL, and MPU-6050 SCL	I2C clock line
Arduino 5 V to LCD VCC and PCA9685 VCC	Logic power for display and servo driver
Arduino 3.3 V to MPU-6050 VCC	IMU power
Arduino GND to LCD GND, PCA9685 GND, and MPU-6050 GND	Shared ground
External 5 V supply positive to PCA9685 V+	Servo power
External 5 V supply negative to PCA9685 GND	Servo power ground

Table 3: Servo channel mapping.

PCA9685 Channel	Controlled Fin
Channel 0	Top fin
Channel 1	Right fin
Channel 2	Bottom fin
Channel 3	Left fin

Table 4: MPU-6050 wiring.

MPU-6050 Pin	Arduino Connection
VCC	3.3 V
GND	GND
SDA	A4 or SDA
SCL	A5 or SCL
INT	Not connected
AD0	Not connected
XDA	Not connected
XCL	Not connected

C Geometry and CAD Measurements

Table 5: Measured geometry values.

Measurement	Value	Description
Fin face area	$6619.915\,\mathrm{mm}^2$	Area of one fin face
Fin face area	$0.006619915\,\mathrm{m}^2$	Same area converted to SI units
CG-to-fin-center distance	106.961 mm	Distance from airframe center to fin center
CG-to-fin-center distance	0.106961 m	Same distance converted to SI units
Inner radius	66.417 mm	Airframe inner radius
Outer radius	68.60 mm	Airframe outer radius
Wall thickness	2.183 mm	Airframe wall thickness
Outer diameter	137.20 mm	Airframe outer diameter

D Control Values

Table 6: Main software values used in the final code.

Variable	Value	Purpose
α	0.97	Complementary filter weight
K	1.8	Control gain
Deadband	1.2°	Reduces twitch near level
Servo center	90°	Neutral fin position
Servo range	50° to 130°	Mechanical motion limit
Max correction	40°	Maximum fin correction from center
Accel range	$\pm 4g$	MPU-6050 accelerometer setting
Gyro range	$\pm 500^\circ/\mathrm{s}$	MPU-6050 gyroscope setting
Smoothing weights	0.8 and 0.2	Smooths pitch and roll commands

E Testing Procedure

1. Verified the PCA9685 servo driver by running a four-servo test command.

2. Verified that the LCD and PCA9685 worked on the shared I2C bus.
3. Verified that the MPU-6050 appeared on the I2C bus at address $0x68$.
4. Confirmed that pitch and roll values changed when the IMU was tilted.
5. Calibrated the IMU by holding the system level during startup.
6. Tested the Serial Monitor commands for zeroing, centering, debugging, and servo testing.
7. Confirmed that the four fins moved in response to IMU motion.

F Additional Images

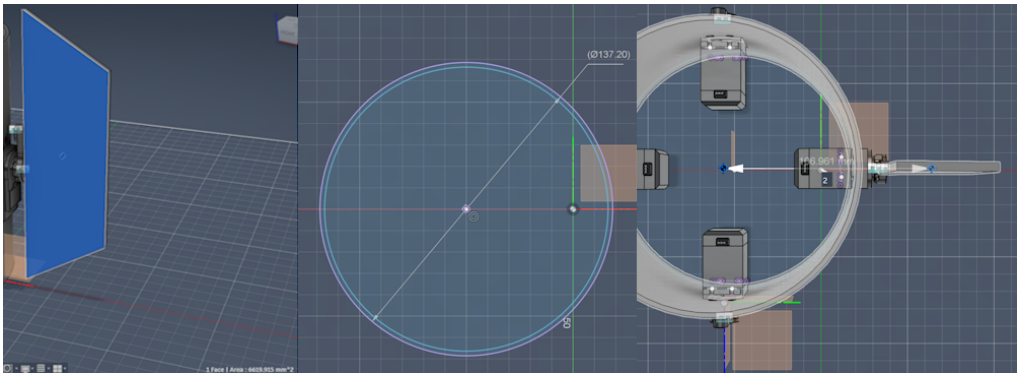


Figure 2: CAD measurements used to document fin area, airframe size, and fin location.